

Çalışma Zamanı Uygulama Öz Koruması (RASP)

CEN429 Güvenli Programlama — Hafta 6

Dr. Öğr. Üyesi Uğur CORUH · 23.10.2026

Bugünün planı (3 saat)

Saat	Bölüm	Konu
1	0–2	Temel kavramlar · RASP nedir · RASP mimarisi
2	3–7	Bütünlük · hata ayıklayıcı · emülatör · kanca/Frida · bellek koruması
3	8–13	Kök/imza · kontrol akışı bütünlüğü · tepki politikası · sınırlar · proje

Kısa tarihçe — uygulama kendini korur

- **1980'ler** — crack/anti-crack kültürü: **anti-debug** ve self-check kökeni
- **2000'ler** — DRM ve mobil bankacılık korumayı uygulamanın **içine** taşır
- **2012** — Gartner **RASP** terimini ortaya atar
- **bugün** — **OWASP MASVS-RESILIENCE** bunu denetlenebilir gereksinime çevirir

RASP yeni bir fikir değil: **MATE saldırganına** verilen kurumsal cevabın adı.

Bu hafta nereye oturuyor?

- 4–5. hafta: kodu **statik** sağladık (derleme, gizleme).
- Bu hafta (6): program **çalışırken** kendini korur → RASP.
- 9/11/14: gizleme ve whitebox RASP ile birlikte çalışır.

Öğrenme çıktısı

Bu hafta **ÖÇ.3** üstünedir.

Sonunda yapabileceğiniz:

- RASP denetimlerini (bütünlük, anti-debug, ortam, kanca) tanımak
- Tepki politikası ve cihaz bağlama tasarlamak
- RASP'in **sınırlarını** ve etik çerçevesini açıklamak

Ana fikir

RASP = **Runtime Application Self-Protection**: uygulamanın çalışırken kendini **izlemesi** ve tehdide **tepki** vermesi.

Algıla → savun → caydır. Ama tek denetim değil, **katman**.

0. Temel kavramlar (sıfırdan)

Çalışma anı (runtime) nedir?

- **Çalışma anı:** programın **çalıştığı** an (derleme değil).
- RASP korumaları burada devreye girer: program kendini **çalışırken** izler.

RASP nedir?

- **RASP (Runtime Application Self-Protection):** uygulamanın çalışırken kendini koruması.
- Kurcalama, hata ayıklama, sahte ortam gibi tehditleri **algılar** ve **tepki** verir.

Hata ayıklayıcı (debugger)

- **Debugger:** programı adım adım çalıştırıp durduran, belleği okuyan araç (gdb, lldb).
- Saldırgan bununla akışı izler, değer değiştirir.
- RASP "bana debugger bağlı mı?" diye bakar.

Emülatör ve sanal makine

- **Emülatör/VM:** bir cihazı **taklit eden** yazılım ortamı (Android emülatör, QEMU).
- Saldırgan analizini gerçek cihaz yerine burada yapar (daha kolay).
- RASP sahte ortamı sezmeye çalışır.

Kanca (hook) ve enstrümantasyon

- **Kanca (hook):** bir fonksiyonun çağrısını **araya girip** değiştirme.
- **Enstrümantasyon:** çalışan programa kod enjekte edip davranışını izleme/değiştirme.
- Araç: **Frida** (çok yaygın dinamik enstrümantasyon aracı).

LD_PRELOAD

- **LD_PRELOAD:** Linux'ta bir kütüphaneyi programdan **önce** yükletip fonksiyonları değiştirme yolu.
- Saldırgan bununla kritik fonksiyonları **kancalayabilir**.
- RASP bunu tespit etmeye çalışır.

Bütünlük ve self-hashing

- **Bütünlük (integrity):** kodun/dosyanın **değişmemiş** olması.
- **Self-hashing:** programın **kendi kodunun** özetini hesaplayıp beklenenle karşılaştırması.
- Kurcalanmışsa özet tutmaz.

Self-hashing — şema

Bütünlük denetimi (self-hashing)

uygulama kendi baytlarını doğrular



Tek başına yetmez: denetimin kendisi de yamalanabilir → örtüşen denetim gerekir.

Özet (checksum/hash)

- **Özet:** bir veriden hesaplanan sabit boyutlu **parmak izi** (SHA-256).
- Veri değişirse özet değişir.
- Bütünlük denetiminin temeli.

Kök (root) / jailbreak

- **Kök (root):** cihaz üzerinde tam yetki (normalde kısıtlı).
- Köklü cihazda korumalar zayıflar; saldırgan her şeye erişir.
- RASP "cihaz köklü mü?" diye bakar.

İmza doğrulama

- Uygulama paketleri **dijital imzayla** imzalanır.
- **İmza doğrulama:** çağıran/yüklenen bileşenin imzasının beklenen olup olmadığını denetleme.
- Sahte/değiştirilmiş bileşeni yakalar.

Kontrol akışı bütünlüğü (sayaç)

- Kritik denetimler **tek bir if** ile yapılırsa, tek nokta yamayla atlanır.
- **Kontrol akışı sayacı**: denetimlerin doğru **sırayla** geçtiğini sayarak doğrulama.
- Tek yama yetmez hale gelir.

Tepki politikası (response)

- **Tepki politikası:** RASP bir tehdit görünce **ne yapacak?**
- Sessizce kapan, işlevi kısıtla, sunucuya bildir, gecikmeli tepki...
- "Hemen çök" her zaman en iyisi değildir.

Cihaz bağlama ve caydırma

- **Cihaz bağlama:** verilerin/anahtarların yalnız **belirli cihazda** anlamlı olması.
- **Caydırma (deterrence):** saldırıyı zahmetli/riskli kılıp vazgeçirme.
- RASP'in nihai amacı: maliyeti yükseltmek.

Şimdi hazırız

Terimler:

çalışma anı · RASP · debugger · emülatör/VM · hook/Frida · LD_PRELOAD · bütünlük/self-hashing ·
özet · kök · imza doğrulama · kontrol akışı sayacı · tepki politikası · cihaz bağlama · caydırma

Şimdi: RASP nedir, ne yapar?

1. RASP nedir?

WAF ↔ RASP — şema

WAF ile RASP farkı

'kurcalandım mı, beni kim izliyor, hangi cihazdayım?'

WAF

uygulamanın ÖNÜNDE
HTTP isteklerini süzer

uygulamanın içini bilmez
sunucu tarafı

RASP

uygulamanın İÇİNDE
kendi kodunu, belleğini,
ortamını izler

MATE saldırganına cevap

RASP, cihazın güvenilmez olduğu durumlar için tasarlanmıştır.

WAF ↔ RASP karşılaştırması

Koruma	Nerede durur	Neyi görür
Ağ güvenlik duvarı	Ağ kenarında	Paketler
WAF (Web App Firewall)	Uygulamanın önünde	HTTP istekleri
RASP	Uygulamanın içinde	Kendi kodu, belleği, ortamı

WAF "dışarıdan gelen istek zararlı mı?" sorar; RASP "**ben** kurcalandım mı?" sorar.

Saldırgan modeli: MATE

- **MATE (Man-At-The-End):** uygulamanın **çalıştığı uç noktanın sahibi** olan saldırgan. Belleği okur, kodu değiştirir, hata ayıklayıcıyla izler. 11. haftadaki **beyaz kutu** modeliyle aynıdır.
- **Acı gerçek:** cihazın sahibi saldırgansa, **hiçbir istemci tarafı koruma nihai olarak kırılmaz değildir.**
- O hâlde RASP neden var? Amaç **kırılmazlık değil**; saldırıyı **ölçeklenemez ve pahalı** kılmak.

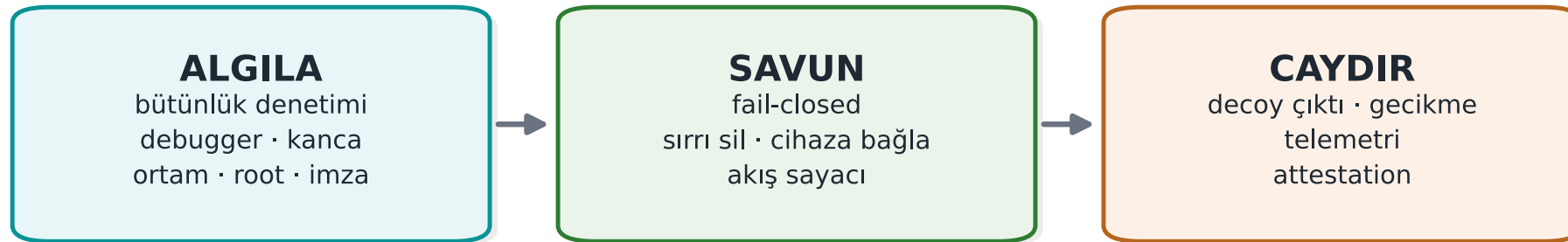
Üç iş: algıla, savun, caydır

- **Algıla:** anormal ortamı/davranışı sez (debugger, emülatör, kurcalama).
- **Savun:** tehdit varsa işlevi kısıtla/durdur.
- **Caydır:** saldırıyı zahmetli kıl → saldırgan vazgeçsin.

RASP döngüsü — algıla → savun → caydır

RASP döngüsü: algıla → savun → caydır

koruma uygulamanın İÇİNDE çalışır



Tek denetim değil, birbirini kontrol eden ÖRTÜŞEN AĞ → tek yama yetmez.

RASP neyi tamamlar?

- Statik korumalar (4–5. hafta) **çalışmadan** geçerli.
- Gizleme (9/11/14) **anlamayı** zorlaştırır.
- RASP, program **çalışırken** aktif savunma ekler.

Gizleme RASP'ı gizler; RASP gizlemeyi çalışırken korur.

RASP neden gerekli?

- Beyaz kutu saldırgan programı **çalıştırıp** izler (debugger, Frida).
- Statik gizleme bunu tek başına durdurmaz.
- RASP "beni izliyorlar mı?" diye bakıp tepki verir.

Ana kural (baştan)

RASP tek başına aşılmaz değildir; **katmanlı** ve **çeşitli** denetimlerle güç kazanır.

Tek bir "root mu?" denetimi kolayca atlanır; onlarca farklı denetim birlikte zorlaşır.

2. RASP mimarisi

Denetim ne zaman çalışır?

- **Başlangıçta:** açılışta ortam denetimi.
- **Kritik işlem öncesi:** ödeme/anahtar kullanımından hemen önce.
- **Periyodik:** arka planda düzenli.
- **Rastgele:** öngörülemez anlarda (saldırgan zamanlayamasın).

Denetim nerede çalışır?

- **Native tarafta** (C/C++): tersine mühendisliği zor, tercih edilir.
- **Yönetilen tarafta** (Java): daha kolay okunur, ama gerekli.
- **Çapraz denetim**: iki taraf birbirini denetler.

Çapraz denetim fikri

- Java tarafı, native kütüphanenin **özetini** denetler.
- Native taraf, Java tarafının **bütünlüğünü** denetler.
- Saldırgan **ikisini birden** atlamak zorunda.

Çok noktalılık ve sahte parametreler

- **Çok noktalılık:** aynı denetim kodun **birçok yerinde**, her seferinde biraz farklı biçimde yapılır; bir nokta yamalanınca diğerleri hâlâ çalışır.
- **Korunan fonksiyonun içinde:** değerli bir fonksiyon (ör. anahtar türetme) kendi giriş noktasında **birkaç denetimi kendisi** yapar — sahada **on beşe yakın** denetime kadar çıkabilir.
- **Sahte parametreler:** giriş noktasında, okunabilirlik için gizlenen ve bir bütünlük denetimini tetikleyen ek parametreler bulunabilir (4. hafta).

Denetim nasıl çalışır? (gizli)

- Denetim kodu **gizlenir** (9. hafta): saldırgan denetimi kolayca bulamasın.
- Sonuç **opak** (düz true/false değil).
- Tepki **gecikmeli** olabilir (birazdan).

Zayıf tasarım — tek karar noktası

```
if (hata_ayiklayici_var() || root_var() || butunluk_bozuk())  
    return HATA;          /* bu tek satırı değiştiren  
                           her şeyi atlatır */  
anahtari_coz();
```

Tek bir karşılaştırmayı ters çeviren saldırgan **bütün korumayı** geçer.

Güçlü tasarım — veri bağımlılığı

Denetim sonucu bir `if` 'e değil, **sonraki hesabın verisine** karışır:

- Denetim sonuçları **anahtar türetmeye** katılır (temizse doğru anahtar, değilse farklı ama geçerli görünen anahtar).
- Denetim sonuçları **opak değerlerle** taşınır (4. hafta).
- Tepki, tetikleyiciden **zaman ve kod olarak uzak** bir yerde verilir.

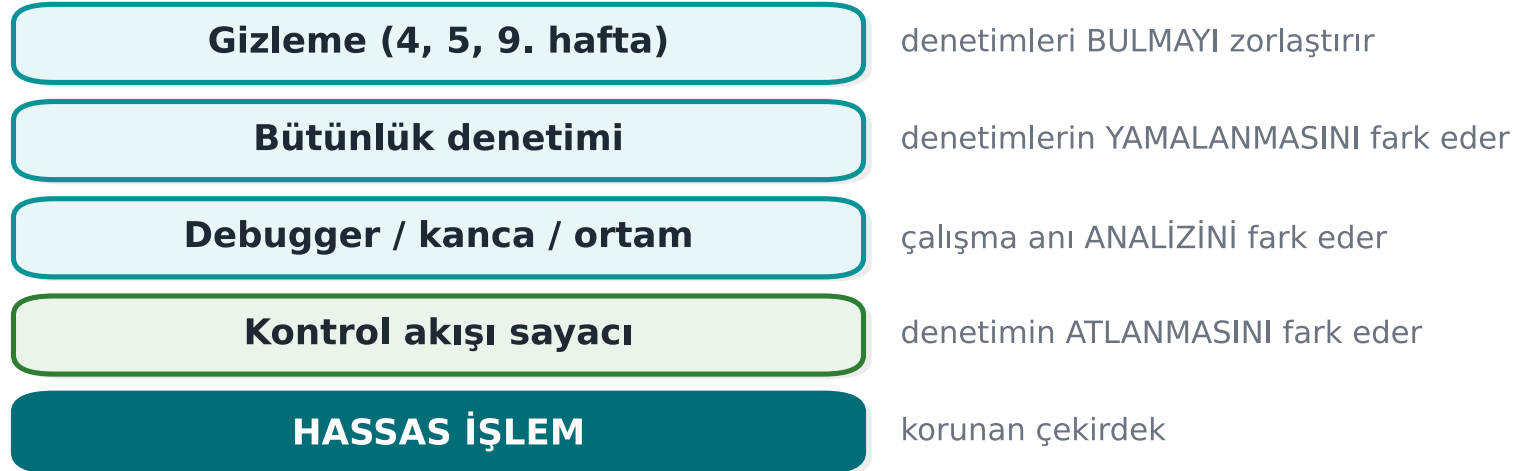
Katmanlı denetim ağı

- Tek denetim değil, **onlarca** farklı denetim.
- Birbirini denetleyen **ağ** (checker network).
- Birini atlamak diğerini tetikler.

Katmanlar birbirini korur — şema

Katmanlar birbirini korur

aynı hassas işlemi saran katmanlar

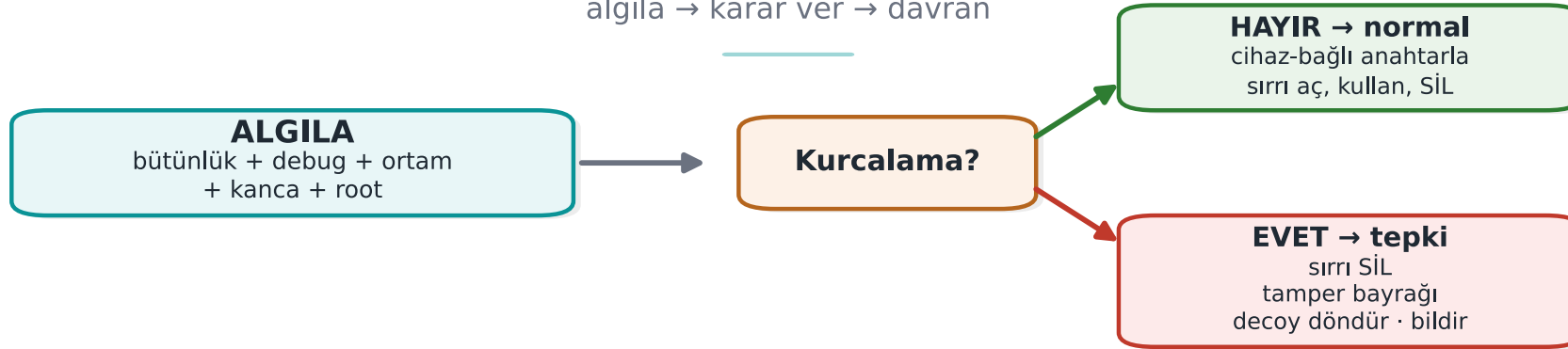


Saldırganın hepsini aynı anda aşması gerekir; bu da saldırı potansiyelini yükseltir.

Mimari · özet şema

RASP motoru: her kritik işlemde

algıla → karar ver → davran



Tepki gecikmeli/sessiz olmalı: hemen çökmek saldırgana denetimin yerini gösterir.

Bu bölümün kuralı (1–2)

Denetimi **tek yerde, tek anda, tek if** ile yapmayın.

Çok noktalı, çapraz, gizli ve veri bağımlı denetimler **birlikte** bir RASP mimarisi kurar.

Bölüm 1–2 — kısa sinama

1. RASP'in üç işi?
2. Çapraz denetim neden güçlü?
3. Neden tek denetim değil, ağ?

Bölüm 1–2 — cevaplar

1. **Algıla** (kurcalama/debug/emülatör), **Savun** (anahtar/veri koruma, işlevi kısıtla), **Caydır** (tepki: kapan/boz/bildir).
2. Denetimler **birbirini** kontrol eder; saldırgan birini kapatınca diğeri fark eder → tek noktayı susturmak yetmez.
3. Tek denetim **tek yamayla** atlanır; örtüşen ağ her düğümü ayrı kırmayı gerektirir → **saldırı potansiyelini** yükseltir.

3. Bütünlük denetimi (self-hashing)

Fikir

- Program, **kendi kod bölümlerinin** özetini hesaplar.
- Beklenen özetle karşılaştırır.
- Kurcalanmışsa özet **tutmaz** → tepki.

Nasıl çalışır? · adım adım

1. Derleme sonrası, kritik kod bloklarının özeti **kaydedilir**.
2. Çalışırken program aynı blokların özetini **yeniden** hesaplar.
3. Kayıtlı özetle karşılaştırır.
4. Fark varsa → kurcalama → tepki.

Adım 1 · özetini kaydet

- Derleme **bittikten sonra**, kritik kod/veri bölgesi belirlenir.
- Bu bölgenin özeti hesaplanır ve **altın (golden) değer** olarak saklanır.
- Bu değer, "kurcalanmamış hâlin" imzasıdır.

Adım 2 · çalışırken yeniden hesapla

- Program **çalışırken**, kayıtlı olan aynı kod/veri bölgesinin özetini **yeniden** hesaplar.
- Bu hesaplama derleme anında değil, **çalışma anında** olur.
- Hesaplama bir kez değil, **periyodik** de yapılabilir.

Adım 3 · karşılaştıır

- Yeni hesaplanan özet, kayıtlı **altın değerle** karşılaştırılır.
- Karşılaştırma **sabit zamanlı** yapılmalı (`memcmp` değil): düz karşılaştırma ilk farklı baytta durup zamanlama sızdırır (3. hafta).
- Eşitse: bütünlük **tamam**.

Adım 4 · tepki ver

- Özet **tutmuyorsa** → kurcalama (yama) algılanmıştır.
- Sonuç doğrudan bir `if` 'e değil, bir **tepkiye** bağlanır (10. bölüm).
- "Hemen çök" yerine daha akıllı bir tepki tercih edilir.

Neyi yakalar?

- İkili dosyaya yapılan **yama** (patch).
- Bir denetimi atlamak için değiştirilen baytlar.
- Enjekte edilmiş kod.

Demo 1 · çalışma zamanı bütünlük denetimi

`code/week-06/01-butunluk-hmac` — self-hashing, HMAC-SHA-256.

- Ders kitabı (Viega & Messier, Tarif 12.2) bunu **CRC32** ile yapar; **CRC32 kriptografik değildir**, saldırgan CRC'yi eşleyecek şekilde kolayca ayarlar. Biz **HMAC-SHA-256** kullanıyoruz (gizli anahtar gerekir).
- Program kendi ikili dosyasının özetini hesaplayıp **altın değer** olarak kaydeder.
- İkili dosyanın bir **kopyası** alınır, bir baytı değiştirilir (yama simülasyonu); yamalı kopyanın özeti altın değerle karşılaştırılır.

Demo 1 · özün kodu

```
/* Kendi yolunu bul, dosyayı oku, HMAC hesapla */
oz_yol(yol, sizeof(yol));
dosya_oku(yol, &veri, &boy);
kripto_hmac_sha256(RASP_ANAHTAR, 32, veri, boy, ozet);

/* sabit zamanli karsilastirma */
unsigned char fark = 0;
for (int i = 0; i < 32; i++)
    fark |= beklenen[i] ^ ozet[i];
if (fark == 0) { /* TAMAM */ } else { /* YAMA */ }
```

Demo 1 · gerçek çıktı

```
ADIM 2 - Normal calisma: butunluk dogrulanir (yama yok)
Beklenen   : bae498a6a982279a8...ee4d01e6
Hesaplanan : bae498a6a982279a8...ee4d01e6
SONUC: BUTUNLUK TAMAM - hedef degismemis.
```

```
ADIM 3 - Saldiri: kopyanin 1 bayti degistiriliyor
ADIM 4 - Yamali kopyanin HMAC'i karsilastiriliyor
Beklenen   : bae498a6a982279a8...ee4d01e6
Hesaplanan : 05fb42748cb5b573...131c6705
SONUC: YAMA ALGILANDI - hedef degistirilmis!
```

Tek bir baytı değiştirmek özeti **tamamen** değiştirdi (çığ etkisi).

Self-hashing tek başına yeterli değil

- Saldırgan, **özet fonksiyonunu** bulup her zaman "tutuyor" dedirtebilir.
- Ya da donanım kesme noktalarıyla denetimi atlar (van Oorschot vd.).
- Bu yüzden: gizle, çeşitlendir, çapraz denetle.

Güçlendirme

- Özet denetimi kodu **gizli** (9. hafta).
- **Birden çok** blok, örtüşen bölgeler.
- Sonuç düz karşılaştırma değil, **hesaba karışır** (denetim sonucu programın davranışını etkiler).

Örtüşen denetim

- Bir denetim başka bir denetimin kodunu da kapsar.
- Saldırgan birini kapatınca diğeri **bozulur**.
- Tek noktadan atlamak zorlaşır.

Bu bölümün kuralı (3)

Kritik kod/veri bölgelerinin bütünlüğünü çalışma zamanında **kriptografik** bir özetle (HMAC/imza) doğrulayın, CRC ile değil.

Denetimi tek noktada ve yalnız başlangıçta yapmayın; **periyodik, çoklu ve örtüşen** yapın.

4. Hata ayıklayıcı algılama

Anti-debug yolları — şema

Hata ayıklayıcı algılama yolları

hiçbiri tek başına kesin değil — sinyaller birlikte tartılır

İşletim sistemi bayrağı

Linux /proc/self/status TracerPid · Windows IsDebuggerPresent

PEB / NtGlobalFlag

Windows süreç yapısındaki izler

Zamanlama

iki nokta arası süre eşiği aşıyor mu?

Kendi kendini izleme

ptrace(PTRACE_TRACEME) başarısızsa biri izliyor

Her sinyal atlatılabilir; güç, birden çok bağımsız sinyalin birlikteliğinden gelir.

Neden debugger tehlikeli?

- Saldırgan programı **durdurup** belleği okur, değer değiştirir.
- Denetimleri tek tek atlayabilir.
- RASP "bana debugger bağlı mı?" diye bakar.

Algılama yolları (kavram)

- İşletim sisteminin "izleniyorum mu?" bilgisini sorma.
- Belirli hata ayıklama arayüzlerinin durumu.
- Zamanlama: bir işlem beklenenden **çok uzun** sürdüyse, biri adım adım çalıştırıyor olabilir.

Platforma göre gösterge · neler eskidi

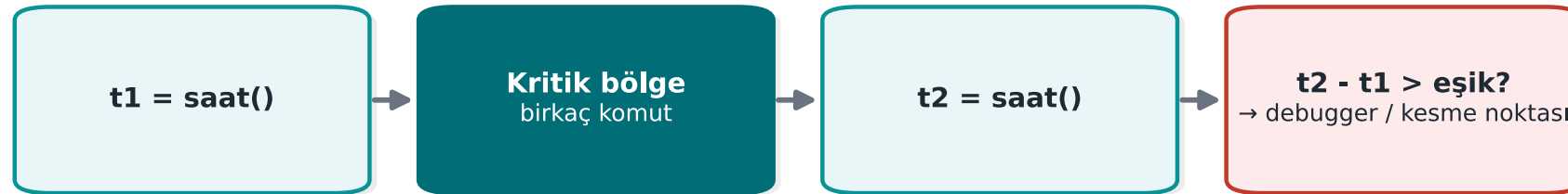
Ortam	Gösterge
Linux / WSL	<code>/proc/self/status</code> → <code>TracerPid</code> (>0 ise izleniyoruz); ana sürecin adı <code>gdb/strace</code> mı
Windows	<code>IsDebuggerPresent</code> (PEB <code>BeingDebugged</code>); <code>CheckRemoteDebuggerPresent</code> ; PEB <code>NtGlobalFlag</code>

2003'ten bugüne: SoftICE öldü, yerine x64dbg/WinDbg/gdb/lldb/**Frida** geçti; `ptrace` + **TracerPid** hâlâ geçerli.

Zamanlama tabanlı sezme

Zamanlama tabanlı sezme

debugger kodu yavaşlatır



Yanlış pozitif riski: yavaş cihaz, arka plan yükü. Eşiği ölçerek seçin.

Debugger adımlaması işi **yavaşlatır**.

Demo 2 · hata ayıklayıcı algılama

`code/week-06/02-hata-ayiklayici` — anti-debug, yalnız okuma.

- Program birden çok bağımsız sinyale bakar, **hiçbir şeyi değiştirmez**.
- Kendini `ptrace` ile de izlemez; yalnız `/proc` ve PEB alanlarını **okur**.
- `demo.sh` programı önce normal, sonra `gdb` altında çalıştırır.

```
/* /proc/self/status icinden TracerPid oku */  
long tp = tracer_pid();  
if (tp > 0) supheli++; /* bir surec bizi izliyor */
```

Demo 2 · gerçek çıktı (WSL)

ADIM 1 - Normal calisma (hata ayiklayici YOK)

1) /proc/self/status TracerPid : 0 (izleyen yok)

2) Ana surec (parent) adi : sh

SONUC: Temiz - izleyen bir arac gorunmuyor.

ADIM 2 - gdb altinda calistir

1) /proc/self/status TracerPid : 2909 (IZLENİYOR)

SONUC: Hata ayiklayici/izleme ARACI algilandi (2 sinyal).

Hata ayıklayıcı yokken TracerPid **0**; gdb altında **2909** (>0) çıktı.

Tespit–karşı-tespit yarışı

Anti-debug klasik bir **silah yarışıdır**: saldırgan `ptrace` 'i `LD_PRELOAD` ile no-op yapabilir, `IsDebuggerPresent` 'in dönüşünü yamalayabilir, PEB bayrağını temizleyebilir.

- Her tek yöntem bilinir ve **atlanabilir**.
- Güç: **birçok** farklı yöntem + gizleme + çeşitlendirme.
- Amaç yine **caydırma** ve **gecikme**.

Tepki nasıl?

- Hemen çökme her zaman iyi değil (saldırgana "burada denetim var" der).
- Daha iyi: **gecikmeli**, **dolaylı** tepki (birazdan bölüm 10).

Bu bölümün kuralı (4)

Anti-debug'ı **tek bir** `if (IsDebuggerPresent()) exit();` olarak yazmayın.

Birden çok bağımsız sinyal toplayın, sonucu bir davranışa bağlayın, tepkiyi **geciktirin**.

Bölüm 3–4 — kısa sinama

1. Self-hashing neyi yakalar?
2. Neden tek başına yetmez?
3. Zamanlama nasıl debugger'ı ele verir?

Bölüm 3–4 — cevaplar

1. **Kod/ikili baytlarının değişmesini** (yama, breakpoint/kod enjeksiyonu) — çalışırken kendi özetini hesaplayıp beklenenle karşılaştırır.
2. Saldırgan hash rutinini/beklenen değeri bulup **atlar** veya karşılaştırmayı NOP'lar; ayrıca yalnız **statik** değişikliği yakalar. Örtüşen denetim gerek.
3. Debugger/kesme noktası kodu **yavaşlatır**; iki nokta arası süre eşiği aşarsa hata ayıklama sezilir.

5. Ortam algılama: emülatör/VM

Emülatör yanlış pozitif — şema

Emülatör algılamada yanlış pozitif

suçsuz kullanıcıyı engellemek en pahalı hatadır

Zayıf sinyaller

pil yok · sensör yok
hipervizör biti

WSL2 / Hyper-V / VBS yüzünden
MEŞRU makinede de görülür

Doğru yaklaşım

birden çok sinyali TART
sertliği derecelendir

sunucu tarafı attestation
ikili 'çalış/çalışma' değil

Bir skor üretin; eşiği ürünün risk iştahına göre ayarlayın.

Neden emülatör?

- Saldırgan analizini gerçek cihaz yerine **emülatörde** yapar.
- Emülatörde durdurmak, izlemek, sıfırlamak kolaydır.
- RASP "gerçek cihazda mıyım?" diye bakar.

Algılama ipuçları (kavram)

- Emülatöre özgü donanım/sensör eksiklikleri.
- Tipik sahte tanımlayıcılar.
- Gerçek cihazda beklenen özelliklerin yokluğu.

Algılama teknikleri (somut)

Teknik	Nasıl	Not
CPUID hipervizör biti	<code>CPUID.1:ECX[31]</code> — "hypervisor present"	Bare-metal'de 0
Hipervizör satıcı imzası	CPUID yaprak <code>0x40000000</code> → 12 karakter	Bazı hipervizörler gizler
Zamanlama	Bir işlemi ölç; ağır emülasyon çok yavaşlatır	Eşik makineye göre değişir

Demo 3 · VM/emülatör ve zamanlama

`code/week-06/03-ortam-zamanlama` — yalnız CPU komutu ve saat okur.

```
(A) CPUID.1:ECX[31] hipervizor biti : VAR
    Hipervizor satıcı imzası       : (gizli/bos)
(B) 2.000.000 işlem süresi       : 0.806 ms
SONUC: Hipervizor GORULDU.
```

Bu çıktı **gerçek** bir WSL2 makinesinde alındı.

En önemli ders: yanlış pozitif

- Yukarıdaki çıktı **hipervizör "VAR"** dedi.
- Ama bu bir analiz ortamı **değil** — sıradan bir geliştirici makinesi!
- Modern Windows'ta **Hyper-V, WSL2, VBS** yüzünden çoğu makine hipervizör bildirir.

Yanlış pozitifin bedeli

- Bu bit tek başına "saldırgan analiz ediyor" anlamına **gelmez**.
- Bir uygulama sırf hipervizör gördüğü için çalışmayı **reddetseydi**, milyonlarca **meşru** kullanıcıyı engellerdi.
- Meşru kullanıcı: geliştirici, kurumsal VM kullanıcısı, WSL2/Hyper-V/VBS'li sıradan Windows kullanıcısı.
- RASP daima **birden çok göstergeyi tartar** ve her birinin **atlatılabileceğini** kabul eder.

Kural: risk skoru, ikili karar değil

- Ortam algılamayı bir **risk skoru** olarak kullanın.
- Birden çok göstergelyi **birleştirin**.
- Nihai kararı **sunucu tarafı risk motoruyla** birlikte verin.

Sahada nasıl uygulanır?

- Mobil dünyada karşılığı **emülatör algılamadır**: `ro.kernel.qemu` , sahte sensörler, tipik IMEI/seri değerleri.
- Aynı ilke geçerlidir: sinyal, **kanıt değil**.
- Kombinasyon + sunucu tarafı denetim.

Bu bölümün kuralı (5)

Emülatör/VM algılamada **tek bir bit asla "saldırı" anlamına gelmez.**

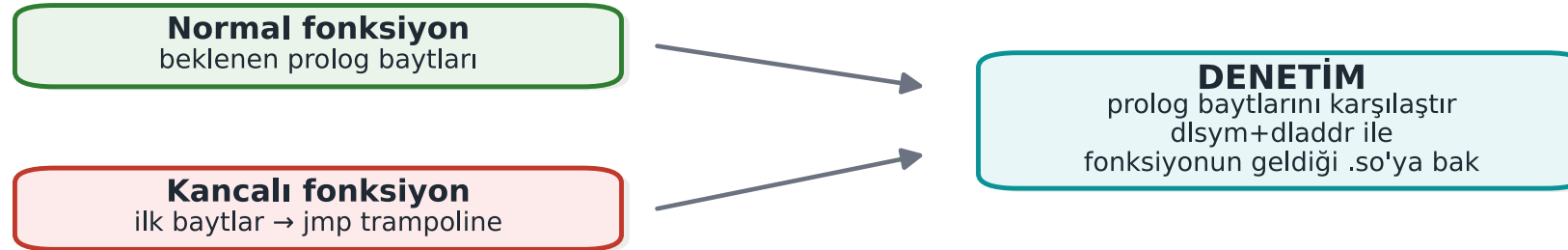
Sert tepki yerine risk skoru; sonuç sunucu tarafı doğrulamayla birlikte değerlendirilir.

6. Kanca ve enstrümantasyon algılama

Kanca algılama — şema

Kanca (hook) nasıl algılanır?

kanca, fonksiyonun ilk baytlarını değiştirir



libc/vDSO dışında bir modülden geliyorsa kanca vardır (LD_PRELOAD).

Kanca (hook) tehdidi

- Saldırgan kritik bir fonksiyonu **kancalar**: çağrıyı araya girip değiştirir.
- Örnek: "imza geçerli mi?" fonksiyonunu her zaman "evet" döndürtmek.
- Araç: **Frida**, Xposed.

LD_PRELOAD kancası

- Linux'ta bir kütüphaneyi programdan önce yükletip fonksiyonları **değiştirme**.
- Kritik fonksiyon saldırganın sürümüyle değişir.
- RASP yüklenen kütüphaneleri denetler.

Kanca teknikleri — tablo

Teknik	Nasıl	Platform
LD_PRELOAD	<code>.so</code> 'yu önyükleyip fonksiyon değiştirme	Linux
PLT/GOT kancası	Çağrı tablosu girişini yönlendirme	Linux/ELF
Satır içi (inline) kanca	İlk baytları <code>jmp</code> ile ezme	Her yer
Frida / Xposed	Dinamik ikili enstrümantasyon	Mobil/masaüstü

Algılama yolları (kavram)

- Beklenmeyen yüklenmiş kütüphaneler/modüller.
- Bilinen enstrümantasyon araçlarının izleri (bellekte, portlarda).
- Fonksiyon adreslerinin beklenenden **sapması**.

Zayıf yol: getenv

```
if (getenv("LD_PRELOAD") == NULL) printf("temiz\n");
```

getenv 'in kendisi de **kancalanıp** NULL döndürebilir. Bu yüzden **kaynağı doğrulamak** gerekir:

1. Kritik fonksiyonu **çöz** (`dlsym`) → 2. **geldiği modülü** bul (`dladdr`) → 3. **meşru** mu denetle → 4. **karar** ver.

Adım 1 · fonksiyonu çöz

```
void *p = dlsym(RTLD_DEFAULT, "time");
```

- `dlsym` , çalışma anında **hangi fonksiyonun** çağrılacağını bulur.
- Önyüklü bir kanca varsa, `dlsym` **onu** döndürür (manipüle edilmiş sonuç).

Adım 2 · kaynağı bul

```
Dl_info info;  
dladdr(p, &info); /* fonksiyonu saglayan .so */
```

- `dladdr` , bir adresin **hangi paylaşımlı nesneden (.so)** geldiğini söyler.
- `getenv` sahteciliğine **dayanmaz** — doğrudan bellek adresine bakar.

Adım 3 · meşru mu denetle

```
int kanca = !mesru_mi(info.dli_fname);
```

- Beklenen kaynak **libc**, **vDSO** ya da dinamik yükleyicidir.
- Yabancı bir **.so** 'dan geliyorsa → **kanca var**.

Adım 4 · karar ver

- Meşru kaynak listesinde **yoksa** → kanca bayrağı kaldırılır.
- Sonuç tek başına bırakılmaz; **tepki politikasına** bağlanır (10. bölüm).
- Birden çok fonksiyon için **tekrarlanır** (tek fonksiyon yetmez).

Fonksiyon bütünlüğü

- Kritik fonksiyonun ilk baytları beklenen mi?
- Kanca genelde başa bir **atlama** yerleştirir → baytlar değişir.
- Değişmişse → kanca var.

Demo 4 · LD_PRELOAD kancası — gerçek çıktı

ADIM 1 - Normal çalışma (LD_PRELOAD yok)

time -> linux-vdso.so.1

getenv -> /lib/x86_64-linux-gnu/libc.so.6

SONUC: Temiz - preload/kanca görünmüyor.

ADIM 2 - Saldırı: sahte kanca YALNIZ bu surece yukleniyor

time -> bin/linux/libsahtekanca.so (KANCA!)

time(NULL) dondurdu : 1234567890 (sabit sahte deger)

SONUC: Fonksiyon kancasi / preload ALGILANDI (2 sinyal).

Temiz koşuda `time`, çekirdeğin **vDSO**'sundan çözüldü — bu **meşrudur**, kanca değil.



Kanca algılama da atlanabilir

- Gelişmiş araçlar izlerini gizler.
- Güç: çoklu yöntem + gizleme + sunucu tarafı denetim.

Bu bölümün kuralı (6)

Kanca algılamada `getenv(LD_PRELOAD)` gibi **manipüle edilebilir** kaynaklara güvenmeyin.

Kritik fonksiyonların **kaynağını** (`dlsym` + `dldaddr`) doğrulayın; birden çok fonksiyonu kontrol edin.

7. Dinamik bellek koruması

Bellek koruma katmanları — şema

Dinamik bellek koruması: hangi katman neyi kapatır?

bellek saldırganının en kolay ulaştığı yerdir

Döküm kapatma

PR_SET_DUMPABLE=0 — çökme dökümünde sır kalmaz

Bellek kilidi

mlock — sır takas alanına yazılmaz

Kısa ömür + silme

explicit_bzero — pencere daralır

Bütünlük sayacı

korunan değer yanında sağlama tutulur; değişirse yakalanır

İzleme tespiti

ptrace / okuma denemesi algılanır ve puana katılır

Hiçbiri tek başına yetmez; her biri saldırganın bir yolunu pahalılaştırır.

Sorun

- Hassas değerler (anahtar, sayaç) bellekte durur.
- Saldırgan belleği okuyup/değiştirip denetimi atlayabilir.

Saldırgan bellekle ne yapar?

Saldırı	Örnek	Hedef
Okuma	Süreç belleğinin dökümünü almak, bilinen değeri aramak	Anahtar, PIN, çözülmüş veri
Değiştirme	Bir sayacı/bayrağı bellekte değiştirmek	Mantık ve denetimler
İzleme	Bir değer değiştiğinde haber veren araçlar	Değişkenlerin yerini keşfetmek

Hepsi için saldırganın süreç belleğine **erişmesi** gerekir (debugger, OS arayüzü, enjekte kütüphane).

Korunan sayaç fikri

- Kritik bir sayacı düz tutmak yerine:
 - bir **gölge kopya** + **özet** ile birlikte tut.
 - her okuma/yazmada tutarlılığı denetle.

Korunan değer · kavram

```
typedef struct { uint32_t deger; uint32_t golge; uint32_t ozet; } Korunan;  
/* okuma: deger == ~golge ve ozet doğru mu? */
```

Saldırgan yalnız `deger` 'i değiştirirse tutarsızlık yakalanır.

Bellek izleme tespiti

- Hassas bölgelere beklenmedik erişim izleniyor mu?
- Bellek dökümü/tarama izleri.
- Amaç: canlı bellek analizi maliyetini artırmak.



Bellek koruması sınırlı

- Kararlı saldırgan yine erişebilir.
- Amaç: **geciktirmek**, anahtarı kısa ömürlü kılmak (10. hafta).

RASP'in sınırları — şema

RASP'in sınırları: ne vaat eder, ne etmez?

cihazın sahibi saldırgansa yeterli zamanla her katman aşılır

ETMEZ

kırılmazlık
MATE'ye karşı kalıcı engel
kötü mimariyi kurtarma
meşru kullanıcıyı ayırt etme garantisi

EDER

saldıryı geciktirir
toplu kırılmayı zorlaştırır
telemetriyle görünürlük
riskli ortamda kapsamı daraltır

Etik sınır: kullanıcının cihazına zarar vermeyin, veri toplamayı şeffaf ve ölçülü tutun.

Bu bölümün kuralı (7)

Bellekte **az, kısa ve dağınık** tutun; kritik değeri **gölge kopya + özetle** koruyun.

Tutarsızlık bir tepkiye bağlanmalı — sessizce yok saymak korumayı boşa çıkarır.

Bölüm 5–7 — kısa sinama

1. Emülatör algılamada neden yanlış pozitif riski var?
2. Kanca (hook) fonksiyon baytlarını neden değiştirir?
3. Korunan sayaç neyi yakalar?

Bölüm 5–7 — cevaplar

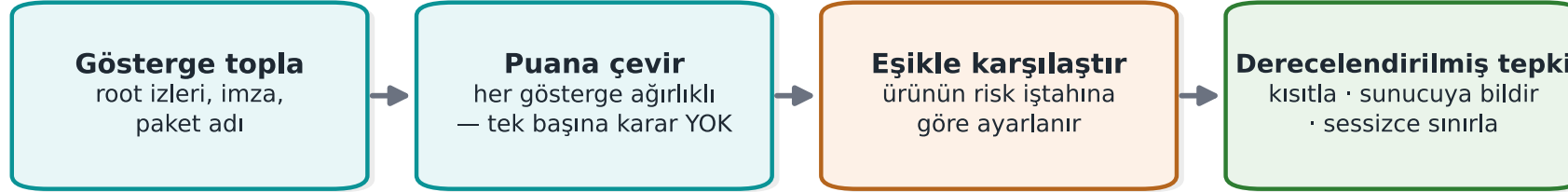
1. Meşru kullanıcı da **VM/emülatör/CI** kullanabilir; zayıf sinyaller gerçek cihazda da görülür → **suçsuzu engelleme** riski. Sertliği derecelendir.
2. Hook, hedefin **ilk baytlarını** bir atlama (jmp/trampoline) ile değiştirir ki çağrı saldırgan koduna sapsın → **prolog baytları** beklenenden farklı olur.
3. **Kontrol akışının atlanmasını**: kritik denetimler çalıştıkça sayaç artar; beklenen değere ulaşmazsa bir denetim **baypas** edilmiştir.

8. Kök ortam ve imza doğrulama

Kök göstergesinden karara — şema

Kök/ayrıcalıklı ortam: göstergeden karara

tek göstergeyle engellemek meşru kullanıcıyı vurur



Root'lu cihaz suçlu değildir; yalnızca güvence düzeyi düşüktür. Tepkiyi ona göre ölçekleyin.

Köklü cihaz neden risk?

- Kök (root) = tam yetki.
- Korumalar zayıflar; saldırgan belleğe, dosyaya, sürece erişir.
- RASP "cihaz köklü mü?" diye bakar.

Kök algılama (kavram)

- Bilinen kök araçlarının/dosyalarının varlığı.
- Normalde kısıtlı olan işlemlerin yapılabilmesi.
- Sistem bütünlüğü göstergeleri.

Kök algılama atlanabilir

- Kök gizleyiciler (root hiding) vardır.
- Tek gösterge yetmez; **birçok** gösterge + sunucu denetimi.
- Tepki: kritik işlevi kısıtla, tümünden engelleme (kullanıcı mağdur olmasın).

Bileşen imza doğrulaması

- Çağırılan/yüklenen bileşenin **dijital imzası** beklenen mi?
- Sahte/değiştirilmiş bir uygulama kütüphaneyi çağırıyorsa yakala.
- Örnek: yalnız beklenen imzalı ana uygulama SDK'yı kullanabilsin.

İmza + özet birlikte

- İmza: bileşen **kim** tarafından yayımlandı?
- Özet: bileşen **değişti mi**?
- İkisi birlikte sahtekârlığı zorlaştırır.

Demo 6 ve 7 · gerçek çıktılar

```
# Demo 7 (kök/ayrıcalık göstergesi)
Ayrıcalık seviyesi : normal kullanıcı
[BULUNDU] çıktı/sahte_su (ek işaret)
SONUC: Ayrıcalıklı/riskli ortam GOSTERGESI var.
```

```
# Demo 6 (bileşen imzası)
Modül imzası TUTMADI -> YENİDEN PAKETLENMİŞ.
Yükleme REDDEDİLDİ.
```

Her iki demo da **yalnız okuyarak/doğrularak** çalışır; sistem değiştirmez.

Bu bölümün kuralı (8)

Kök/ayrıcalık göstergeleri **kanıt değil, sinyaldir** — kolayca sahtelenir.

Dinamik yüklenen her bileşeni yüklemekten **önce** kriptografik olarak doğrulayın; mümkünse karşılıklı doğrulama kurun.

9. Kontrol akışı bütünlüğü

Tek `if` neden yetmez?

```
if (imza_gecerli()) devam(); /* tek nokta */
```

- Saldırgan bu tek dalı **yamalar** → denetim atlanır.
- ix. haftadaki opak boolean + rastgele çıkış burada kritik.

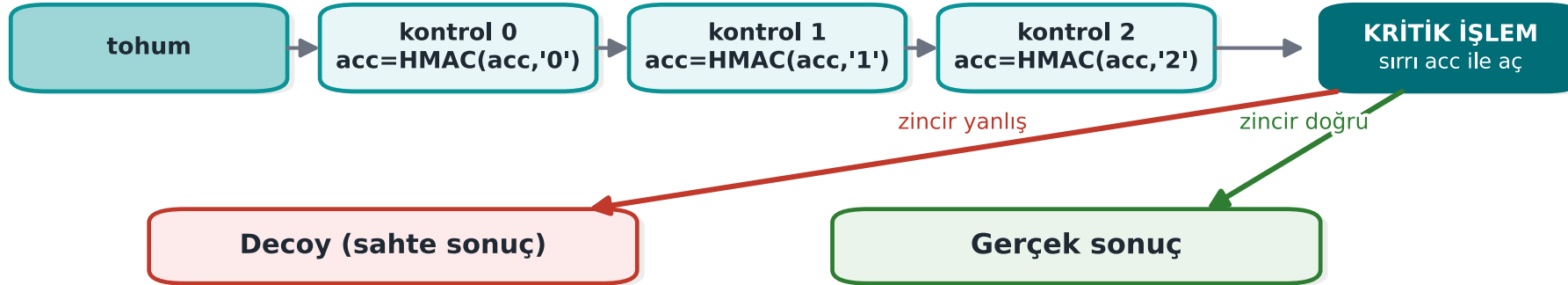
Kontrol akışı sayacı

- Kritik denetimler doğru **sırayla** geçilmeli.
- Her denetim bir **sayacı** günceller.
- Sonda sayaç beklenen değilse → biri atlandı.

Kontrol akışı sayacı — şema

Kontrol akışı sayacı: denetim atlandı mı?

her denetim akümülatörü ilerletir



Tek `if` bir NOP'la atlanır; sayaç, denetimin GERÇEKTEN çalıştığını sonradan kanıtlar.

Çift sayaç

- İki bağımsız sayaç (biri artan, biri azalan).
- Toplamları/ilişkileri sabit olmalı.
- Saldırgan **ikisini birden** tutarlı değiştirmek zorunda.

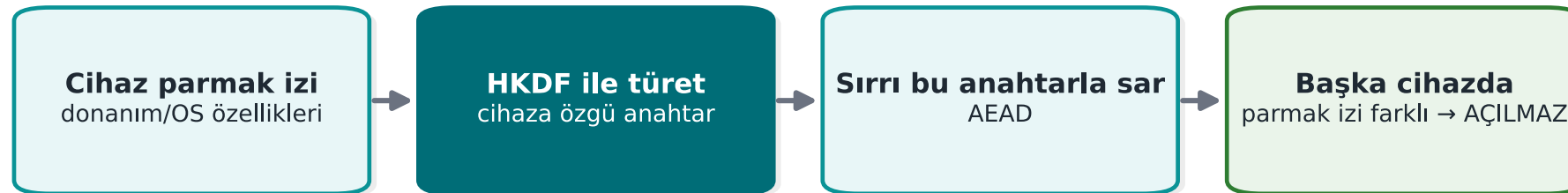
Sonucu davranışa bağla

- Denetim sonucu düz bir `if` değil.
- Sonuç, bir sonraki adımın **hesabına** girer (ör. bir anahtar türetmesine).
- Atlanırsa program **yanlış** çalışır (sessizce bozulur).

Cihaz bağlama — şema

Cihaz bağlama: kopyalama işe yaramasın

çalınan dosya başka cihazda anlamsızdır



Code lifting ve kopyalama saldırılarının ölçeklenmesini bu engeller (11. hafta).

Demo 5 · skip saldırısı — gerçek çıktı

```
SENARYO 1 - Normal: kontroller sirayla calisir  
SONUC: ODEME ONAYLANDI -> "ODEME-ONAYI-TOKEN-4242"
```

```
SENARYO 2 - Saldiri: kontroller tamamen ATLANIR  
      (uyari: sayac=0 iz=0x0 beklenen=3/0x7)  
SONUC: ODEME REDDEDILDI (zincir anahtari yanlis). decoy doner
```

```
SENARYO 4 - Saldiri: kontroller YANLIS SIRADA calisir  
SONUC: ODEME REDDEDILDI (zincir anahtari yanlis). decoy doner
```

Kontroller atlanınca ya da sırası bozulunca zincir farklı çıktı; **gerçek sonuç üretilmedi.**

Bu bölümün kuralı (9)

Kritik kararları **tek bir boolean**'a bağlamayın (yamalanır).

Güvenlik kontrollerini bir **kontrol akışı sayacına** ve mümkünse bir **veri bağımlılığına** bağlayın.

10. Tepki politikası ve caydırma

"Hemen çök" neden kötü?

- Saldırgana "işte tam burada denetim var" der.
- Denetimi bulup atlamayı kolaylaştırır.
- Daha akıllı tepkiler gerekir.

İyi tepki örnekleri

- **Gecikmeli:** tehdidi sonra, başka yerde tetikle (kaynağı gizle).
- **Dolaylı:** işlevi sessizce bozar (yanlış sonuç üret).
- **Sunucuya bildir:** risk skoru artır, işlemi sunucuda reddet.

Tepki stratejileri — tablo

Strateji	Ne yapar
Fail-closed	Şüphede işlemi reddet, güven deposuna düşme
Sırrı sil	Tamper anında değerli veriyi güvenle sil
Decoy (sahte) çıktı	Çökmek yerine rastgele/sahte sonuç döndür
Gecikmeli/örtük tepki	Tepkiyi zaman/mesafe olarak tetikleyiciden ayır
Cihaz/sürüm bağlama	Sırrı cihaza/sürüme bağla
Telemetri/attestation	Olayı sunucuya bildir; sunucu risk motoru karar versin

Tepki tasarımı

- Kullanıcı deneyimini bozma (yanlış pozitif riski).
- Sert tepki yalnız **yüksek güvenli** göstergede.
- Çoğu durumda: kısıtla + sunucu denetimi.

Algıla → karar → tepki döngüsü

Demo 8 (`code/week-06/08-tamper-yanit`) bu döngüyü tek bir **öz koruma motorunda** birleştirir:

1. Bir dizi kontrol **çalışır** (bütünlük, anti-debug, ortam...).
2. Sır, cihaz-bağlı bir anahtarla (HKDF + AES-GCM) **sarılıdır**.
3. Tamper algılanınca motor sırrı **siler**, bayrak kaldırır, **decoy** döndürür.

Adım A · algıla (senaryo 1 — normal)

```
SENARYO 1 - Normal: kontroller gecer, cihaz dogru -> sir acilir  
SONUC: TEMIZ. Sir acildi ve islem yapiliyor  
-> "ODEME-ANAHTARI-7C4A"  
(sir kullanildiktan sonra bellekten guvenle silindi)
```

Kontroller **geçti**, cihaz doğruydı → sır açıldı, kullanıldı, hemen **silindi**.

Adım B · karar (senaryo 2 — tamper)

```
SENARYO 2 - Tamper: bir kontrol basarisiz -> sil + decoy + bayrak  
SONUC: TAMPER ALGILANDI -> algilama kontrolu basarisiz  
Politika: sir silindi, tamper bayragi kaldırildi,  
olay kaydedildi.  
Cokmek yerine SAHTE (decoy) sonuc dondu: 60797327...
```

Bir kontrol **başarısız** oldu → motor **karar** verdi: sil + kaydet + decoy.

Adım C · tepki (senaryo 3 — başka cihaz)

```
SENARYO 3 - Baska cihaz: kontroller gecer ama  
             cihaz anahtari tutmaz  
SONUC: TAMPER ALGILANDI -> cihaz/surum baglama tutmadi  
      (klonlama?) ... decoy doner
```

Kontroller geçse **bile** cihaz-bağlı anahtar tutmadı → sır **açılmadı**. Bu, Hafta 3'teki güvenlik kabuğunun en iç katmanıdır.

Cihaz bağlama

- Anahtar/veri yalnız **belirli cihazda** anlamlı.
- Kopyalanırsa (başka cihaz) işe yaramaz.
- Kod taşımaya (9/11. hafta) karşı.

Cihaz bağlama akışı · adım adım

Adım 1 — parmak izini oku: cihazın donanım/OS parmak izi okunur (5. hafta).

Adım 2 — anahtar türet: parmak izi + sürüm dizesinden **HKDF** ile bir anahtar türetilir; sır bu anahtarla **AES-GCM** ile sarılır.

Adım 3 — taşınırsa aç: dosya başka bir cihaza kopyalansa bile, o cihazın parmak izi **farklı** olduğundan türetilen anahtar tutmaz → sır **açılmaz**.

Caydırma · nihai amaç

RASP saldırıyı **imkânsız** kılmaz; **zahmetli ve riskli** kılar.

Yeterince katman + sunucu denetimi → saldırgan vazgeçer ya da yakalanır.

Bu bölümün kuralı (10)

Tepkiyi tetikleyiciden **ayırın**: hemen `exit()` saldırıgana yol gösterir.

Tamper anında sırrı **silin**, çökmek yerine **decoy** döndürün, olayı sunucuya bildirin, sırrı cihaza bağlayın.

Bölüm 8–10 — kısa sinama

1. Köklü cihaz neden risk?
2. Tek `if` 'i sayaç neden güçlendirir?
3. Neden "hemen çök" kötü bir tepki olabilir?

Bölüm 8–10 — cevaplar

1. **Root/jailbreak** bellek okuma/yazma, hook, sertifika enjeksiyonu engellerini kaldırır → RASP varsayımları ve izolasyon çöker.
2. Tek `if` bir NOP/yama ile atlanır; **sayaç** denetimin gerçekten çalıştığını sonradan doğrular (dağıtık kanıt) → tek yama yetmez.
3. "Hemen çök" saldırılarına denetimin **tam yerini** gösterir ve yanlış pozitifte meşru kullanıcıyı vurur. **Gecikmeli/sessiz/aldatıcı** tepki daha iyi.

11. Sınırlar ve etik

RASP'in sınırları

- Her tek denetim **atlanabilir**.
- Köklü cihazda güç azalır.
- Yanlış pozitif → gerçek kullanıcı mağdur olabilir.
- Bakım maliyeti yüksek (araçlar gelişir).

O yüzden RASP...

- Katmanlı (çok denetim), çeşitlendirilmiş, gizli.
- Sunucu tarafı denetimle birlikte (asıl güvence).
- Tek başına "aşılmaz" iddiası **yanlıştır**.

Etik ve kullanıcı

- RASP kullanıcının cihazında çalışır; **aşırı veri toplama** etik değil.
- Yanlış pozitif kullanıcıyı hizmetten edebilir → dikkatli tepki.
- Şeffaflık ve gizlilik dengesi.



Güvenli örnek kuralı

- Ders demoları öğrenci cihazına **zarar vermez**.
- Gerçek kök/sistem değişikliği yok; sentetik göstergeler.
- Amaç kavramı öğretmek, saldırı aracı yapmak değil.

12. Proje: bu hafta (S10)

Proje · S10 (RASP + tepki)

- [] En az iki farklı RASP denetimi (ör. bütünlük + anti-debug).
- [] Denetimlerin **ne zaman/nerede** çalıştığını belgele.
- [] Tepki politikası: tehdit görülünce ne olur?
- [] Cihaz bağlama kararı ve gerekçesi.
- [] Sınır ve kalan risk (neyi korumaz).

Değerlendirici gözüyle

- "RASP var" yetmez; kaç katman, nasıl gizli, nasıl tepki?
- Sunucu denetimiyle nasıl birleşiyor?
- Yanlış pozitif riski nasıl yönetiliyor?

Çözümlü kendini sinama

Soru 1

RASP'in üç işi nedir?

Cevap: Algıla (anormal ortam/kurcalama), savun (işlevi kısıtla/durdur), caydır (maliyeti yükselt).

Soru 2

Self-hashing neyi yakalar, neden tek başına yetmez?

Cevap: İkiliye yapılan yamayı/kurcalamayı yakalar. Saldırgan özet fonksiyonunu bulup atlatabildiği için tek başına yetmez; gizleme + çapraz + örtüşen denetim gerekir.

Soru 3

Çapraz denetim neden güçlüdür?

Cevap: Java native'i, native Java'yı denetler; saldırgan **ikisini birden** atlamak zorunda kalır.

Soru 4

Zamanlama tabanlı anti-debug nasıl çalışır?

Cevap: Küçük bir iş beklenenden çok uzun sürdüyse, biri programı adım adım çalıştırıyor (debugger) olabilir.

Soru 5

Tek **if** denetimini kontrol akışı sayacı neden güçlendirir?

Cevap: Denetimlerin doğru sırayla geçtiğini sayar; tek bir dalı yamalamak sayacı bozar, atlanma yakalanır.

Soru 6

"Hemen çök" neden her zaman iyi bir tepki değildir?

Cevap: Saldırgana denetimin tam yerini gösterir. Gecikmeli/dolaylı tepki + sunucu bildirimi daha iyidir.

Soru 7

Emülatör algılamada yanlış pozitif riski neden önemli?

Cevap: Gerçek cihaz/geliştirici ortamı emülatör gibi görünebilir; sert tepki gerçek kullanıcıyı mağdur eder.

Soru 8

RASP'in temel sınırı nedir?

Cevap: Her tek denetim atlanabilir; köklü cihazda zayıflar. Güç katmandan, çeşitlilikten ve sunucu denetiminden gelir.

Quiz-1 tarzı örnek sorular

Örnek · çoktan seçmeli

RASP çapraz denetiminde amaç nedir?

- A) Kodu küçültmek
- B) İki tarafı da atlamayı zorunlu kılmak ✓
- C) Şifreleme hızını artırmak
- D) Günlüğü kapatmak

Örnek · kısa cevap

Bir bütünlük denetimini güçlendiren üç yöntem yazın.

Gizleme · çoklu/örtüşen blok · çapraz denetim · sonucu davranışa bağlama.

Sözlük

Terim	Anlam
RASP	Çalışma anı öz-koruma
MATE	Uçtaki saldırgan (cihazın sahibi)
Self-hashing	Kendi kodunun özetini denetleme
Hook/Frida	Fonksiyon çağrısını araya girip değiştirme
CFI sayacı	Denetim sırasını sayarak doğrulama
Cihaz bağlama	Veriyi belirli cihaza (HKDF ile) bağlama
Decoy	Çökmek yerine dönen sahte sonuç
Fail-closed	Şüphede işlemi reddetme
Attestation	Bütünlüğün sunucu tarafı kanıtı

Özet: bu haftanın tek cümlesi

RASP program çalışırken kendini izler ve tehdide tepki verir; ama tek denetim değil, **katmanlı, çeşitlendirilmiş, gizli** denetimler ve **sunucu tarafı** doğrulama ile güç kazanır.

Gelecek hafta

7. hafta — Ara proje gösterimi (RAP1) ve 8. hafta — Quiz-1

Vize dönemi: projenizin ilk yarısını gösterip Quiz-1'e hazırlanın.

Ek A · Bir RASP katmanı kurmak

Hedef

Sentetik bir "lisans denetimi"ni RASP ile koruyalım:

- bütünlük denetimi
- hata ayıklayıcı denetimi
- akıllı tepki

Adım 1 · korunacak nokta

```
int lisans_gecerli(void) {  
    /* ... kontrol ... */  
    return sonuc;    /* saldırganın hedefi */  
}
```

Bu fonksiyon ve onu çağıran akış kritik.

Adım 2 · bütünlük denetimi ekle

- Derleme sonrası bu bölgenin özetini kaydet.
- Çalışırken yeniden hesapla, karşılaştır.
- Fark → kurcalama bayrağı.

Adım 3 · anti-debug ekle

- Zamanlama + OS göstergesiyle "izleniyor muyum?".
- Sonucu tek başına bırakma; başka denetimle **birleştir**.

Adım 4 · sonucu davranışa bağla

- `lisans_gecerli` sonucu düz true/false değil.
- Bütünlük + anti-debug göstergeleri, sonucun **hesabına** girer.
- Kurcalama varsa fonksiyon **yanlış** çalışır.

Adım 5 · tepki

- Hemen çökme yerine: bayrağı sunucuya bildir, işlevi kısıtla.
- Gecikmeli tetik: kaynağı gizle.

Adım 6 · gizle ve çeşitlendir

- Denetim kodunu gizle (9. hafta).
- Her sürümde farklı yerleştirme (çeşitlendirme).
- Çapraz denetim ekle.

Adım 7 · ölç ve belgele

- Kaç denetim, nerede, hangi tepki?
- Başarım maliyeti (denetimler yavaşlatır).
- S10'a yaz.

Ek B · Mini vaka

Kurgu · ödeme uygulaması

- Yerel anahtar + ödeme akışı.
- Hedef: kurcalama ve canlı analiz zorlaştırsın.

Vaka · denetim yerleşimi

- **Başlangıç:** kök/emülatör/imza denetimi.
- **Ödeme öncesi:** bütünlük + anti-debug + hook denetimi.
- **Periyodik/rastgele:** tekrar.

Vaka · tepki

- Yüksek güvenli gösterge → işlemi reddet + sunucuya bildir.
- Zayıf gösterge → sunucu risk skorunu artır.
- Kullanıcı deneyimini koru (yanlış pozitif).

Vaka · katmanlar

- RASP + gizleme (9) + whitebox (11) + kısa ömürlü anahtar (10).
- Sunucu tarafı: hız sınırı, anomali.
- Bir katman aşılrsa bile diğerleri sürer.

Vaka · kalan risk

- Köklü cihazda kararlı saldırgan ilerleyebilir.
- Ama: çok katman + sunucu denetimi → hasar sınırlı, saldırı pahalı.
- Açıkça yazılır.

Ek C · RASP denetim kataloğu

Katalog (1)

Denetim	Ne yakalar	Sınır
Bütünlük (self-hash)	İkili yama	Özet fn. atlanabilir
Anti-debug	Debugger bağlı	Bilinen yöntemler atlanır
Emülatör	Sahte ortam	Yanlış pozitif

Katalog (2)

Denetim	Ne yakalar	Sınır
Hook/Frida	Fonksiyon kancası	Gizli araçlar
Kök	Yetki yükseltme	Kök gizleyiciler
İmza	Sahte bileşen	Anahtar sızarsa
CFI sayacı	Denetim atlama	Karmaşık yama

Katalogdan ders

- Her denetimin bir **kör noktası** var.
- Tek başına hiçbirisi yeterli değil.
- Katalogdan **birçoğunu** seç, birleştir, çeşitlendir.

Denetim seçimi

- Varlığın değerine göre kaç katman?
- Platforma göre hangi denetimler anlamlı?
- Başarım bütçesi ne kadar?

Kararı ve gerekçesini S10'a yaz.

Ek D · Saldırgan $\leftrightarrow$ savunmacı

Neden bu diyalog?

Güvenlik bir **hamle-karşı hamle** oyunudur.

Her savunmaya bir saldırı, her saldırıya yeni savunma.

Katmanlı savunma bu yüzden gereklidir.

Bütünlük · diyalog

- **Savunma:** self-hashing ekle.
- **Saldırgan:** özet fonksiyonunu bul, hep "tutuyor" dedirt.
- **Savunma:** özeti gizle, örtüştür, sonucu davranışa bağla.

Anti-debug · diyalog

- **Savunma:** debugger denetimi ekle.
- **Saldırgan:** denetimi bul, atla.
- **Savunma:** çok yöntem + zamanlama + gizleme; tepkiyi gecikmeli yap.

Hook · diyalog

- **Savunma:** fonksiyon baytlarını denetle.
- **Saldırgan:** kancayı gizle, baytları geri düzelt.
- **Savunma:** çapraz denetim + sunucu tarafı doğrulama.

Kök · diyalog

- **Savunma:** kök göstergelerini denetle.
- **Saldırgan:** kök gizleyici kullan.
- **Savunma:** çok gösterge + sunucu risk skoru; sert tepki değil kısıtlama.

Diyalogdan ders

- Tek savunma her zaman aşılr.
- Güç: **çokluk + gizlilik + çeşitlilik + sunucu.**
- Amaç: saldırıyı **ekonomik olmaktan çıkarmak.**

Ek E · Tepki karar akışı

"Tehdit gördüm, ne yapayım?" — 1

Soru 1: Gösterge ne kadar güvenilir?

- **Çok güvenilir** → işlemi reddet + sunucuya bildir.
- **Belirsiz** → devam, ama risk skorunu artır.

"Ne yapayım?" — 2

Soru 2: Kullanıcı deneyimi etkilenir mi?

- Yanlış pozitif riski yüksekse → **sert tepki verme**.
- Sessiz kısıtlama + sunucu denetimi tercih.

"Ne yapayım?" — 3

Soru 3: Tepki denetimi ele verir mi?

- Hemen çökme → kaynağı gösterir.
- **Gecikmeli/dolaylı** tepki → kaynağı gizler.

Tepki akışı · tek bakış

Tepki politikası: ne yapmalı?

tepki, algılamadan daha çok düşünülmesi gereken kısımdır

Hemen çok

saldırgana denetimin YERİNİ gösterir — kötü

Sessiz/gecikmeli tepki

sonraki işlemde sırrı sil, bağlantıyı kes

Decoy (sahte sonuç)

saldırgan yanlış yolda zaman kaybeder

Sunucuya bildir

telemetri + hesap/cihaz kısıtlama

Yanlış pozitifte meşru kullanıcıyı vurmayan, saldırganı yormayan tepki seçilir.

Ek F · Hızlı başvuru

Sık yapılan hatalar

- Tek denetime güvenmek.
- Denetimi gizlememek (kolay bulunur).
- "Hemen çök" tepkisi (kaynağı ele verir).
- Sert tepki + yüksek yanlış pozitif (kullanıcı mağdur).
- Sunucu denetimini atlamak (asıl güvence orada).

RASP kontrol listesi

- [] ≥ 2 farklı denetim türü
- [] Denetimler gizli + çeşitlendirilmiş
- [] Çapraz denetim (Java $\leftrightarrow$ native)
- [] Akıllı tepki (gecikmeli/dolaylı)
- [] Cihaz bağlama
- [] Sunucu tarafı doğrulama
- [] Yanlış pozitif planı

Son söz (6. hafta)

RASP, saldırganla oynanan bir **zaman** oyunudur: her katman biraz daha zaman kazandırır.

Asıl güvence: katmanlı RASP + gizleme + kısa ömürlü anahtar + **sunucu denetimi**.